MODULE 5 – VERSION CONTROL

1. Introduction to Version Control

Version Control is a system used to track and manage changes made to files over time.

It helps developers maintain different versions of their source code and collaborate with other developers.

Why do we need Version Control?

- Tracks changes made to source code.
- Maintains the history of a project.
- Allows developers to restore previous versions.
- Supports team collaboration.
- Helps identify who made a particular change.
- Prevents loss of project information.
- Supports parallel development.

Example:

Suppose a developer is working on a Student Management System.

- Version 1: Student registration.
- Version 2: Student login.
- Version 3: Student update functionality.
- Version 4: Student delete functionality.

Version Control helps maintain the history of all these changes.

2. Types of Version Control Systems

There are three major types of Version Control Systems.

1. Local Version Control System

- Stores different versions of files on the local computer.
- Tracks changes within one system.
- Does not provide built-in team collaboration across multiple machines.

2. Centralized Version Control System (CVCS)

- Uses a central server to store project files and history.

- Developers connect to the central server to access the project.
- Requires access to the central server for most version-control operations.

Examples:

- Apache Subversion (SVN).
- Perforce.

3. Distributed Version Control System (DVCS)

- Every developer has a local copy of the repository and its history.
- Developers can work independently.
- Changes can be shared with other repositories.
- Supports flexible collaboration.

Example:

- Git.

3. Introduction to Git

Git is a distributed version control system used to track changes in source code.

It was created by Linus Torvalds in 2005.

Features of Git:

- Distributed version control.
- Tracks file changes.
- Supports branching and merging.
- Maintains commit history.
- Supports offline development.
- Provides collaboration features.
- Helps restore previous versions.
- Supports efficient project management.

Advantages of Git:

1. Fast and efficient.
2. Free and open-source.
3. Supports multiple developers.
4. Maintains complete project history.
5. Allows developers to work independently.
6. Helps manage different project versions.

4. Git vs GitHub

Git:

- Git is a version control system.
- It is installed and used on a computer.
- It tracks changes in files.
- It manages commits, branches and merges.

GitHub:

- GitHub is a cloud-based platform for hosting Git repositories.
- It allows developers to share projects.
- It supports collaboration.
- It provides pull requests, issue tracking and project management features.

Other Git hosting platforms:

- GitLab.
- Bitbucket.
- Azure Repos.

Important Difference:

Git is the tool used for version control, whereas GitHub is a platform used to host and collaborate on Git repositories.

5. Installing and Configuring Git

After installing Git, configure your username and email.

Check Git Version:

```
None
git --version
```

Configure Username:

```
None
git config --global user.name "Your Name"
```

Configure Email:

```
None
git config --global user.email "yourmail@example.com"
```

View Configuration:

```
None
git config --list
```

Explanation:

- git config: Configures Git settings.
- --global: Applies settings to all repositories for the current user.
- user.name: Sets the commit author name.
- user.email: Sets the commit author email.

These values are recorded in commit metadata. They do not by themselves authenticate you with GitHub.

6. Understanding Git Repository

A repository (repo) is a storage location that contains project files and their version history.

Two types of repositories:

Local Repository:

- Stored on the developer's computer.
- Used to manage and track local changes.

Remote Repository:

- Hosted on a remote server or platform.
- Used to share project history and changes.
- Example: A GitHub repository.

A Git repository contains the project's files and Git's internal history information.

7. Creating a Git Repository

To start tracking a project using Git, initialize a repository.

Command:

```
None
git init
```

This creates a hidden `.git` directory in the project folder.

The `.git` directory stores Git repository information, including commits, branches and configuration.

Example:

```
None
mkdir StudentManagement
cd StudentManagement
git init
```

Now the folder is a Git repository.

8. Git Working Areas

Git mainly works with three areas.

1. Working Directory

- The location where developers create and modify files.
- Changes are made here first.

2. Staging Area

- An intermediate area where selected changes are prepared for the next commit.
- Developers choose which changes should be included in a commit.

3. Local Repository

- Stores committed project snapshots and their history.
- Commits are saved here.

Git Workflow:

Working Directory → Staging Area → Local Repository → Remote Repository

9. Git Status

The git status command displays the current state of the working directory and staging area.

Command:

```
None
git status
```

It helps identify:

- Modified files.
- Untracked files.
- Staged files.
- Uncommitted changes.
- Current branch information.

10. Adding Files to the Staging Area

The git add command moves changes into the staging area.

Add a Specific File:

```
None
git add Program.cs
```

Add Multiple Files:

```
None
git add Program.cs README.md
```

Add All Changes:

```
None
git add .
```

The dot (.) represents the current directory and its contents, subject to Git's ignore rules.

11. Git Commit

A commit saves a snapshot of the staged changes in the local repository.

Command:

```
None
git commit -m "Add student registration functionality"
```

Explanation:

- git commit: Creates a commit.
- -m: Specifies the commit message.
- Message: Describes the changes included in the commit.

Advantages of Commits:

- Maintains project history.
- Helps track changes.
- Makes it easier to identify problems.
- Allows developers to restore earlier versions.

Important:

Only staged changes are included in a normal commit.

12. Writing Good Commit Messages

A commit message should clearly explain what changes were made.

Examples:

```
None
git commit -m "Add student registration functionality"
git commit -m "Fix login validation"
git commit -m "Update employee details page"
git commit -m "Add database connection"
git commit -m "Create product management API"
```

Conventional Commit Format:

Conventional Commits follow a standard format:

```
None
type(scope): description
```

Common types:

- feat: Adds a new feature.
- fix: Fixes a bug.
- docs: Updates documentation.
- style: Makes formatting changes without changing functionality.
- refactor: Improves code without changing its external behavior.
- test: Adds or updates tests.
- chore: Performs maintenance tasks.

Examples:

```
None
git commit -m "feat: add student registration"
git commit -m "fix: correct login validation"
git commit -m "docs: update project README"
git commit -m "refactor: improve student service"
git commit -m "test: add student service tests"
git commit -m "chore: update project dependencies"
```

13. Viewing Commit History

The git log command displays the commit history.

Basic Command:

```
None
git log
```

Short Commit History:

```
None
git log --oneline
```

Graph View:

```
None
git log --oneline --graph --all
```

Detailed Commit Information:

```
None
git show
```

Useful information available in commit history:

- Commit ID.
- Author.
- Date.
- Commit message.
- Changes included in the commit.

14. Git Diff

The git diff command displays differences between versions of files.

View Unstaged Changes:

```
None
git diff
```

Shows changes in the working directory that have not been staged.

View Staged Changes:

```
None
git diff --staged
```

Shows changes already added to the staging area.

Compare Two Commits:

```
None
git diff commit1 commit2
```

It helps developers understand exactly what has changed.

15. Connecting a Local Repository to GitHub

To share a local Git project on GitHub, create a remote repository and connect it to the local repository.

Step 1: Create a Repository on GitHub

1. Open GitHub.
2. Sign in to your account.
3. Click New Repository.
4. Enter the repository name.
5. Select public or private visibility.
6. Create the repository.

Step 2: Add the Remote Repository

```
None
git remote add origin
https://github.com/username/repository-name.git
```

Here:

- git remote add: Adds a remote repository.
- origin: Common default name for the remote.
- URL: Address of the GitHub repository.

Step 3: Check Remote Connection

```
None
git remote -v
```

Step 4: Rename the Current Branch to Main

```
None
git branch -M main
```

Step 5: Push the Project

```
None
git push -u origin main
```

The local main branch is uploaded to the remote repository.

16. Git Remote

Remote refers to a version of the repository hosted somewhere other than the local computer.

Common remote commands:

View Remote Repositories:

```
None
git remote -v
```

Add Remote:

```
None
git remote add origin <repository-url>
```

Rename Remote:

```
None
git remote rename origin upstream
```

Remove Remote:

```
None
git remote remove origin
```

Change Remote URL:

```
None
git remote set-url origin <new-repository-url>
```

17. Git Push

The git push command uploads local commits to a remote repository.

Command:

```
None
git push origin main
```

Explanation:

- git push: Uploads commits.
- origin: Remote repository name.
- main: Branch being pushed.

First Push:

```
None
git push -u origin main
```

The `-u` option sets the upstream tracking relationship. Later, you can generally use:

```
None
git push
```

18. Git Pull

The git pull command downloads changes from a remote repository and integrates them into the current local branch.

Command:

```
None
git pull origin main
```

Git pull generally performs two operations:

1. Fetch: Downloads remote changes and updates remote-tracking references.
2. Integrate: Merges or rebases the fetched changes into the current branch, depending on configuration and options.

Why use git pull?

- To get the latest project changes.
- To synchronize a local repository with a remote repository.
- To receive changes made by other team members.

19. Git Fetch

The git fetch command downloads updates from a remote repository without integrating them into the current branch.

Command:

```
None
git fetch origin
```

To view the fetched changes:

```
None
git log --oneline HEAD..origin/main
```

Difference between fetch and pull:

git fetch:

- Downloads remote updates.
- Does not merge them into the current branch automatically.
- Allows developers to inspect changes before integrating them.

git pull:

- Fetches remote updates.
- Integrates them into the current branch.

20. Cloning a Git Repository

Cloning creates a local copy of an existing remote repository.

Command:

```
None
git clone https://github.com/username/repository-name.git
```

This downloads the repository, including its available history and files.

Clone with a Custom Folder Name:

```
None
git clone https://github.com/username/repository-name.git MyProject
```

Clone a Specific Branch:

```
None
git clone -b branch-name
https://github.com/username/repository-name.git
```

Common uses:

- Downloading a team project.
- Starting work on an existing application.
- Accessing source code hosted on GitHub.

21. Branching in Git

A branch is an independent line of development within a repository.

It allows developers to work on different features without directly changing the main development branch.

Example branches:

```
None
main
 ├── feature-login
 ├── feature-registration
 └── bugfix-validation
```

Advantages:

- Supports parallel development.
- Keeps feature development separate.
- Reduces the risk of disturbing stable code.
- Makes collaboration easier.

22. Branch Commands

View Branches:

```
None
git branch
```

Create a Branch:

```
None
git branch feature-login
```

Switch to a Branch:

```
None
git switch feature-login
```

Create and Switch to a Branch:

```
None
git switch -c feature-login
```

Rename a Branch:

```
None
git branch -m old-name new-name
```

Delete a Local Branch:

```
None
git branch -d feature-login
```

Force Delete a Local Branch:

```
None
git branch -D feature-login
```

List Remote Branches:

```
None
git branch -r
```

List All Branches:

```
None
git branch -a
```

23. Git Merge

Merging combines changes from one branch into another branch.

Example:

Suppose a developer completes a login feature in the feature-login branch.

The changes need to be integrated into main.

Commands:

```
None
git switch main
git merge feature-login
```

Explanation:

1. Switch to the target branch.
2. Run the merge command.
3. Git combines the changes from the specified branch into the current branch.

Types of Merge:

1. Fast-Forward Merge:
    - Moves the target branch pointer forward when its history allows it.
    - No merge commit is required.
2. Three-Way Merge:
    - Combines changes using the common ancestor and the tips of both branches.
    - Usually creates a merge commit when a fast-forward is not possible and no other merge strategy is selected.

24. Merge Conflicts

A merge conflict occurs when Git cannot automatically combine changes from different branches.

Example:

- Developer A modifies a line in Program.cs.
- Developer B modifies the same line differently.
- Git may not know which version should be retained.

Git marks the conflicting section:

```
None
<<<<<<< HEAD
Console.WriteLine("Hello");
=======
Console.WriteLine("Welcome");
>>>>>>> feature-login
```

Resolving Merge Conflicts:

1. Open the conflicting file.
2. Examine both changes.
3. Decide which changes should be retained.
4. Edit the file and remove the conflict markers.
5. Test the code.
6. Stage the resolved file.
7. Complete the merge commit when required.

Commands:

```
None
git status
git add Program.cs
git commit
```

The merge commit command is needed when Git has not already completed the merge.

Important:

Merge conflicts are normal in team development. They should be resolved carefully to avoid losing valid changes.

25. Pull Requests (PR)

A Pull Request is a request to merge changes from one branch into another branch on a Git hosting platform such as GitHub.

Pull Request Workflow:

1. Clone the repository.
2. Create a feature branch.
3. Make changes.
4. Stage and commit changes.
5. Push the branch to GitHub.
6. Open a Pull Request.
7. Team members review the changes.
8. Address review comments if necessary.
9. Merge the Pull Request after approval and required checks.

Advantages:

- Supports code review.
- Improves code quality.
- Helps team collaboration.
- Provides a discussion area for changes.
- Helps identify problems before merging.

26. Forking in GitHub

Forking creates your own copy of another user's repository under your GitHub account.

It is commonly used in open-source development.

Fork Workflow:

1. Open another developer's repository.
2. Click Fork.
3. GitHub creates a copy under your account.
4. Clone your fork to your local computer.
5. Create a new branch.
6. Make changes.
7. Commit and push your changes.
8. Create a Pull Request to the original repository.

Difference Between Fork and Clone:

Fork:

- Creates a copy of a repository under your GitHub account.
- Performed on the Git hosting platform.

Clone:

- Downloads a repository to your local computer.
- Performed using Git.

27. Git Stash

Git stash temporarily saves changes that are not ready to be committed.

It is useful when a developer needs to switch branches but wants to keep unfinished work.

Save Changes:

```
None
git stash
```

Save with a Message:

```
None
git stash push -m "Work in progress"
```

View Stashes:

```
None
git stash list
```

Apply the Latest Stash:

```
None
git stash apply
```

Apply and Remove the Latest Stash:

```
None
git stash pop
```

Delete a Specific Stash:

```
None
git stash drop stash@{0}
```

Delete All Stashes:

```
None
git stash clear
```

Difference:

- apply: Applies stashed changes but keeps the stash entry.
- pop: Applies stashed changes and removes that stash entry if successful.

28. Git Reset

The git reset command moves the current branch to another commit and can change the staging area or working directory.

It is commonly used to undo local commits or unstage files.

Soft Reset:

```
None
git reset --soft HEAD~1
```

- Moves the current branch back by one commit.
- Keeps the changes staged.

Mixed Reset:

```
None
git reset --mixed HEAD~1
```

- Moves the current branch back by one commit.
- Keeps the changes in the working directory.
- Unstages those changes.

- This is the default reset mode.

Hard Reset:

```
None
git reset --hard HEAD~1
```

- Moves the current branch back by one commit.
- Resets the staging area and working directory.
- Discards tracked changes that are not present in the target commit.

Important Warning:

`git reset --hard` can permanently discard uncommitted work. Use it carefully.

29. Git Revert

The git revert command creates a new commit that reverses the changes introduced by an earlier commit.

Command:

```
None
git revert commit-id
```

Example:

```
None
git revert a1b2c3d
```

Advantages:

- Preserves existing commit history.
- Creates a new commit to undo previous changes.
- Suitable for undoing changes in shared branches.

Difference Between Reset and Revert:

Reset:

- Moves the current branch pointer.

- Can change or discard local changes.
- Rewrites local branch history.

Revert:

- Creates a new commit that reverses an earlier commit.
- Preserves existing history.
- Safer for shared branches.

30. Git Restore

The git restore command is used to restore files in the working directory or staging area.

Discard Unstaged Changes in a File:

```
None
git restore Program.cs
```

Unstage a File:

```
None
git restore --staged Program.cs
```

This removes the file from the staging area but keeps its working-directory changes.

Important:

Restoring a file can discard changes. Check the file before executing the command.

31. .gitignore File

The .gitignore file tells Git which untracked files or patterns it should ignore.

It is useful for avoiding the upload of unnecessary files.

Common files and folders to ignore:

```
None
bin/
obj/
.vs/
```

```
*.user
*.suo
.env
```

Example .gitignore:

```
None
# Build output
bin/
obj/

# Visual Studio files
.vs/
*.user

# Environment and secret files
.env
```

Why use .gitignore?

- Avoids unnecessary files in the repository.
- Keeps the repository clean.
- Helps prevent accidental tracking of local configuration and secrets.
- Reduces unnecessary project files.

Important:

Adding an already tracked file to .gitignore does not automatically remove it from Git tracking.

To stop tracking an already tracked file while keeping it locally:

```
None
git rm --cached filename
```

32. Git Tags

A tag is a reference that identifies a specific point in the repository's history.

Tags are commonly used to mark software releases.

Example:

```
None
git tag v1.0.0
```

View Tags:

```
None
git tag
```

Create an Annotated Tag:

```
None
git tag -a v1.0.0 -m "First release"
```

Push a Specific Tag:

```
None
git push origin v1.0.0
```

Push All Tags:

```
None
git push origin --tags
```

Delete a Local Tag:

```
None
git tag -d v1.0.0
```

33. Git Cherry-Pick

Cherry-pick applies the changes introduced by a particular commit to the current branch.

Example:

```
None
git switch main
git cherry-pick commit-id
```

Uses:

- Applying a specific bug fix.
- Transferring an individual commit between branches.
- Selecting changes without merging an entire branch.

34. Git Rebase

Rebase is used to move or replay commits onto another base commit.

It helps create a more linear project history.

Example:

```
None
git switch feature-login
git rebase main
```

This reapplies the feature branch commits on top of the current main branch.

Difference Between Merge and Rebase:

Merge:

- Combines branch histories.
- Preserves the original branch structure.
- May create a merge commit.

Rebase:

- Replays commits on a new base.
- Creates new commit IDs for the replayed commits.
- Produces a more linear history.

Important:

Avoid rebasing commits that other developers are already using unless the team has agreed on how to handle the rewritten history.

35. GitHub Collaboration Workflow

In a team project, developers often work on separate features.

Example:

Project: Student Management System.

- Developer A: Student Registration.
- Developer B: Student Login.
- Developer C: Student Profile.
- Developer D: Student Reports.

Typical workflow:

1. A team member clones the repository.
2. Creates a feature branch.
3. Implements the assigned feature.
4. Checks changes using git status and git diff.
5. Stages the required files.
6. Creates a commit.
7. Pushes the feature branch.
8. Creates a Pull Request.
9. Other developers review the changes.
10. The approved changes are merged into the main branch.

This workflow helps developers work on different features independently.

36. GitHub Issues

GitHub Issues are used to track tasks, bugs, enhancements and questions related to a project.

Common uses:

- Report bugs.
- Suggest features.
- Assign tasks.
- Discuss project requirements.
- Track development progress.

Example issues:

- Fix login validation.
- Add student search functionality.
- Improve dashboard design.
- Update database configuration.

37. GitHub README.md

README.md is a Markdown file used to describe a project.

It usually contains:

- Project title.
- Project description.
- Features.
- Technologies used.
- Installation instructions.
- How to run the project.
- API endpoints when relevant.
- Screenshots when helpful.
- Author or contributor information.

Example:

```
None
# Student Management System

## Description
A web application to manage student information.

## Technologies
- C#
- ASP.NET Core
- Entity Framework Core
- SQL Server

## Features
- Add students
- View students
- Update students
- Delete students

## How to Run
1. Clone the repository.
2. Open the solution.
3. Configure the database connection.
```

```
4. Run the application.
```

38. Markdown Basics

Markdown is a lightweight markup language used to format README files and documentation.

Heading:

```
None
# Main Heading
## Section Heading
### Subheading
```

Bold Text:

```
None
**Bold Text**
```

Italic Text:

```
None
*Italic Text*
```

Unordered List:

```
None
- HTML
- CSS
- JavaScript
```

Ordered List:

```
None
1. First
2. Second
```

```
3. Third
```

Link:

```
None
[Google](https://www.google.com)
```

Image:

```
None
![Project Screenshot](image.png)
```

Code Block:

```
None
```csharp
Console.WriteLine("Hello World");
```
```

39. Git Authentication

Git needs authentication and authorization to access private repositories or perform operations that require permission.

Common authentication methods:

1. HTTPS with a Personal Access Token.
2. SSH using an SSH key.
3. GitHub CLI authentication.

Personal Access Token (PAT):

A token used to authenticate Git operations over HTTPS.

Important points:

- Do not share tokens.
- Do not commit tokens to source code.
- Use appropriate permissions.

- Store secrets securely.

SSH:

SSH keys allow secure authentication between your computer and GitHub.

A common SSH setup includes:

```
None
ssh-keygen -t ed25519 -C "yourmail@example.com"
```

Then add the public key to your GitHub account.